

Dilema e agjentit të dyfishtë

- **Kufiri kohor:** 12 minutes.
- **Hapësira e ruajtjes:** 5 GB
- **Mjedisi:** një GPU (≈ 16 GB VRAM), pa internet
- **Madhësia e zgjidhjes:** `solution.ipynb` ≤ 1 MB
- **Rezultati bazë:** 0

Në qendrën kombëtare të AI-së në Astana, dy modele kompjuterike — Modeli R (një ResNet-18) dhe Modeli V (një ViT-Tiny) — po analizojnë fotografi. Aktualisht, të dy modelet po e kryejnë punën në mënyrë të përsosur, duke arritur saktësi 100% dhe duke dhënë të njëjtin rezultat për çdo imazh. Për të testuar se sa të ndryshëm janë në të vërtetë «trutë» e tyre inteligjentë, shkencëtari kryesor ju jep një sfidë: bëni ndryshime të vogla, pothuajse të padukshme, në pikselat e çdo fotografie, në mënyrë që Modeli R dhe Modeli V të mos pajtohen aspak.



1. Detyra

Dy klasifikues të paratrajnuar imazhesh shqyrtojnë të njëjtin imazh. Në imazhet e ofruara në këtë detyrë, të dy klasifikuesit punojnë me saktësi 100%.

- **Modeli R:** `torchvision.models.resnet18` (një CNN, ResNet18).
- **Modeli V:** `vit_tiny_patch16_224` i `timm` (një Transformer, ViT-Tiny).

Detyra juaj është të krijoni një ndryshim të vogël («perturbim») për çdo imazh, në mënyrë që të dy modelet të mos pajtohen. Për çdo imazh, duhet të krijoni **dy perturbime të ndryshme**:

- **Tipi A:** pas shtimit të tij, Modeli R vazhdon ta klasifikojë saktë imazhin, por Modeli V e klasifikon gabim.
- **Tipi B:** pas shtimit të tij, Modeli V vazhdon ta klasifikojë saktë imazhin, por Modeli R e klasifikon gabim.

Çdo perturbim duhet të jetë mjaftueshëm *i vogël* sa të jetë i vështirë për t'u vënë re. Perturbimet më të vogla marrin rezultate më të larta (shihni Seksionin 5). Perturbimi zbatohet drejtpërdrejt mbi imazhin origjinal, në nivel pikselësh.

2. Të dhënat publike

Me detyrën ofrohet një bashkësi imazhesh, e organizuar në dy ndarje — `train` (100 imazhe) dhe `test_public` (100 imazhe) — secila me imazhe me rezolucione të ndryshme. Të gjitha imazhet janë nga 1000 klasat e ImageNet-1K dhe si Modeli R, ashtu edhe Modeli V, arrijnë saktësi 100% në të dyja ndarjet.

Ofrohen skedarët e mëposhtëm:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

Gjatë vlerësimit, dosja juaj `dataset/test_public/` zëvendësohet në mënyrë transparente nga dy bashkësi të fshehura imazhesh (`test_leaderboard_a` dhe `test_leaderboard_b`) për vlerësimin zyrtar. Secila prej tyre përmban **100 imazhe** në formatin PNG dhe një skedar etiketash.

Shënim: Për këtë detyrë, etiketat në dataset-et e testimit janë të qasshme.

3. Format i rezultatit

Për çdo imazh, duhet të prodhoni dy skedarë:

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

- `{index}` (0, 1, 2, ...), përputhet me emrin e imazhit në dataset-e.
- Çdo skedar është një tensor i vetëm, i ruajtur me `torch.save`. Forma e tij duhet të jetë $3 \times H \times W$, ku H dhe W përputhen me rezolucionin **origjinal** të atij imazhi (jo 224×224).

- Kodi duhet të prodhojë vetëm një skedar ZIP, `submission.zip`. Vendosini të gjithë skedarët `.pt` në nivelin më të lartë të arkivit ZIP, pa dosje përmbajtëse ose nëndosje.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

Notebook-u do t'ju njoftojë nëse ka ndonjë problem me formatin e rezultatit.

4. Kufizimet

- **Modelet:** Duhet të përdorni `torchvision.models.resnet18(pretrained=True)` dhe `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Nuk lejohen modele të tjera të paratrajnuara.
- **Pipeline-i i transformimit (zbatohet gjatë vlerësimit):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. Shikoni pjesën 3 të `baseline.ipynb` për detaje.
- **Rezolucioni i perturbimit:** Duhet të përputhet me rezolucionin **original** të imazhit të papërpunuar (jo 224×224). Tensori i shtohet imazhit të papërpunuar *përpara* pipeline-it të transformimit.
- **Formati i rezultatit:** Vetëm skedarë `.pt` — jo PNG/JPG. Tensorët i shtohen imazhit të papërpunuar dhe vlerat e pikselëve kufizohen në `[0, 1]` përpara parapërpunimit.
- **Emërtimi i skedarëve:** Flat-list, me format të rreptë `{index}_a.pt / {index}_b.pt`. Pa nëndosje brenda skedarit zip.
- **Bibliotekat:** `torch`, `torchvision`, `timm`.

5. Vlerësimi

Rezultati përfundimtar llogaritet si më poshtë. Le të jetë M numri i imazheve në ndarje, $Score_A$ numri i perturbimeve të suksesshme të Tipit A dhe $Score_B$ numri i perturbimeve të suksesshme të Tipit B:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF është një funksion i projektuar për të penalizuar perturbimet me normë të lartë dhe për të qenë shumë i ndjeshëm pranë tavanit të performancës. Ai është i kufizuar në intervalin nga 0.5 deri në 1. Implementimi i plotë mund të shihet në Seksionin 8 të `solution.ipynb`.

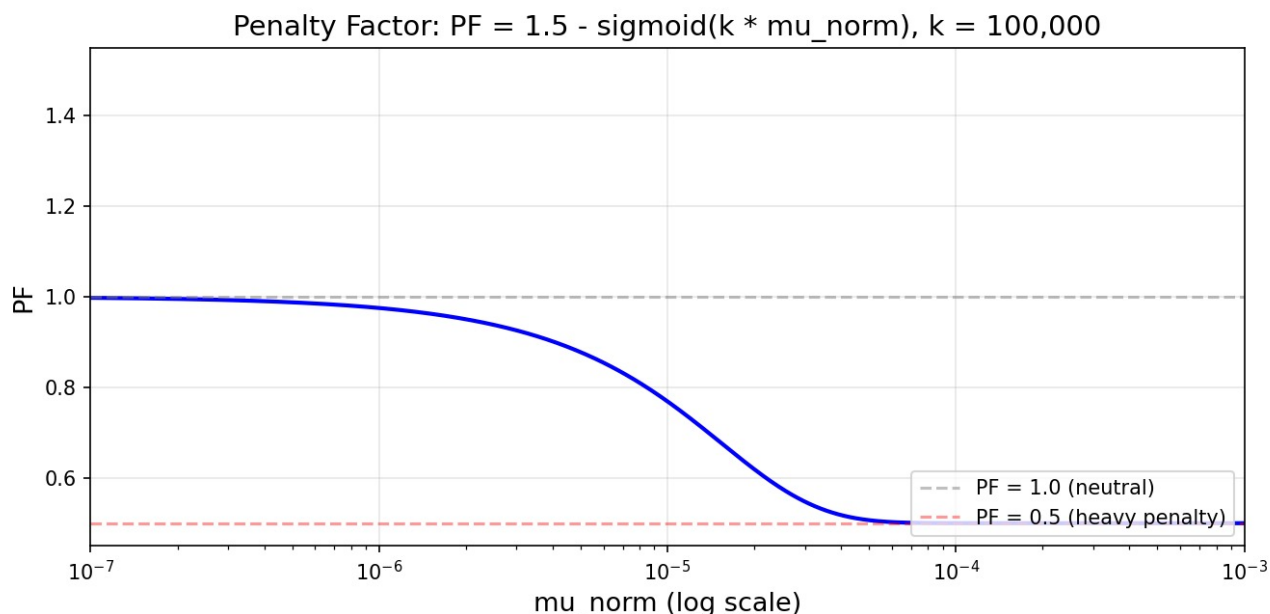


Figura: Kurba e funksionit të penalizimit.

6. Kontrolloni dorëzimin

Në notebook ka kontrolle që ju njoftojnë nëse ka probleme formatimi, në Seksionin 7 të notebook-ut `solution.ipynb`.

7. Testimi lokal

`solution.ipynb` përmban një shembull të plotë dhe funksional. Ai ngarkon të dhënat publike, për dy modelet dhe vlerësuesin zyrtar, si dhe shkruan një skedar ZIP për dorëzim. Lexojeni përpara se të filloni.

8. Si të dorëzoni

- Ruani ndryshimet tuaja në `solution.ipynb`.
- Hapni skedarin Git në shiritin anësor në majtë të JupyterLab.
- Kalojeni `solution.ipynb` në **Stage** (ikona + pranë tij).
- Shkruani një mesazh commit-i dhe klikoni **Commit**.
- Klikoni ikonën e resë me shigjetë lart për të bërë push.
- Kthehuni në këtë faqe të konkursit dhe klikoni **Submit**.

Dorëzoni saktësisht një skedar, të emërtuar `solution.ipynb`.